<Links:

Are these the correct Links? The links in the pdf are not clickable (for me):

https://github.com/ALaDyn/pepc

https://gitlab.jsc.fz-juelich.de/SLPP/pepc/pepc

PDF for the paper?

https://juser.fz-juelich.de/record/29160 Technical report for the algorithm

Team Fulda

AMT: Julia + Dagger.jl

Tree construction (“most important point”)

- We would not favour a (distributed) pointer-based oct-tree. We favour the hashable oct-tree (HOT) with Morton IDs.
- Reason: HOT fits Dagger much better than a distributed pointer tree — Morton keys are globally valid across ranks, so we avoid latency-bound remote pointer-chasing during traversal (a pointer is only meaningful within one rank).
- For an SC paper we also see better acceptance chances, since HOT keeps us comparable to PEPC.
    - Agreed

Data distribution

- Sort / rebuild: local tree rebuild every timestep (cheap, no communication)
- The global sort + rebalance only periodically — could be a parameter (or a load-imbalance trigger)?

- Idea remote data for traversal: bulk LET (Locally Essential Tree) exchange and no fine-grained on-demand fetching? (LET = each rank imports, in one bulk exchange, the remote tree nodes it will need.
    - See my comment below (Sanjay/Charm++)

Task / leaf sizes

- Two separate parameters?
    - leaf-bucket size (bodies per leaf), and
    - task granularity (one task bundles several buckets).

Comparability vs. showcasing AMT strengths

- How strictly do we mandate a common implementation strategy vs. allow each AMT to play to its strengths?
- Our thinking: same data distribution for all AMTs, but tree build / traversal / load-balancing free for each AMT?

Experiments

- Do we focus only on weak scaling, or also strong scaling? What is the target number of nodes?
    - We will focus on both and we aim for 1000is nodes
- Which cluster do we use?
    - Not yet determined. We only know that it will be CPU only and x86 or Arm CPUs.
    - Each time provides a build.sh and run.sh script with the source code. We will use these to compile the code and run the code.
- What exactly do we report as the time? Full step (tree build + traversal + integrate) or per-phase timing? I/O included or excluded?
    - IO will be excluded.
- Metrics beyond wall-time (e.g. interactions/s, parallel efficiency, code-complexity)?

Correctness

- Should the energy / verification tracking be disable-able for measuring best performance?
- Question: is the "softening factor epsilon^2" from (3) also needed in (12)?

Paraview

- Honestly, we have no experience yet with Paraview with Julia. (But google says WriteVTK.jl writes VTK / .pvtu / .pvd, so it should work.)

Reference implementation

- Does anyone provide a reference implementation in sequential C(++)?
    - No, we assumed that the PEPC will be our reference implementation.
    - Since we have no funding for the project, implementing a sequential C++ version is not feasible. Some mentioned that not all of us have experience with C++ and some would prefer a Python version.
- Do we also need a brute-force reference for real correctness checking?
    - No, we do not need that since we have a reference implementation already

Team Kassel

AMT: Itoyori(FBC)

Tree construction

- We lean towards Hashable Oct-Tree, for similar reasons as the Fulda team.

Experiments

- Will we get a login on the test machine so that we can experiment and tune our program there?
    - Most likely not, however, since it is a standard x86 or Arm CPU there should not be much tuning needed. It will most likely be one of the DOE clusters and giving foreign nationals access might not be easy. We can run on a few nodes and send you the results for checking.

Team Charm++

We are ok with Morton order. I wonder if specifically calling out hashable Oct tree is overspecification. I need to think about it more.

- I think the PEPC code calls it hashable and since we want to compare against this reference implementation, we adopted their naming.

I think we should include I/O. How else are we going to ensure everyone has the same data? Is the random data generator a distributed one? Can we make a distributed one?

- This question was more about if we measure IO for the total timing. Yes, we should all write to Paraview for checking and visualizing the data.

Suggestion: time different phases of the algorithm separately, so we can compare performance phase-wise

- Agree

We should include a clustered (high density variations) and non-clustered (sort of like initial conditions of the universe in cosmology simulations) datasets to be benchmarked separately.

Data distribution: replication is not scalable, so NO unless I misunderstood. The common tree above partitioned tree could be replicated but the upward pass (centroids or multipoles) can get complicated if we allow that.

As to team Fulda (Julia)'s comment about remote data during traversal: I think bulk transfer of all needed data at once is against the spirit of task-based runtimes, which specialize in data driven scheduling. For that reason also, I hope we will be doing data dependent fetches (i.e. the opening criteria should not be statically deducible).

Also please remember not to specify the benchmark (and terminology) in terms of physical resources such as cores or nodes. Charm++ virtualizes the notion of processor so the program is expressed in terms of objects (or partitions or tree-pieces in this context).

How are we going to ensure rebalancing is needed? Unless we do a significant number of steps, or introduce means to enhance evolving imbalance, we may require large number of steps to test load balancing.

Especially if we use galaxy-collision dataset, there is an issue of multiple timestepping. Allowing multiple timestepping (where slow moving bodies calculate forces on them less frequently) will complicate the implementations and comparisons significantly. I suggest disallowing that optimization.

New broad comment (7/3/26): Reading the PEPC paper, I am concerned that PEPC is an overspecification for what we need. It makes several design decisions that are either unnecessary for us (as a group) or incompatible with our design options (as Charm++ application). For example,

1. we rely on overdecomposition for a variety of reasons including as a part of our load balancing strategies. This is best done by allowing multiple chares, each a complete oct tree, on each core. Is that compatible?

2. We talk about hashed-oct-tree instead of pointers, but does it preclude using pointers within a process or core (pthread)? That is unnecessary and suboptimal. I'd rather use pointer-based data structures locally.
3. Particles and tree-nodes requested from remote processors: I'd prefer to (have an option to) integrate them into a local pointer based tree, rather than a hashtable .
4. The mechanism for traversal is over-specified in PEPC.
5. Tree build: tree should/need not go down to a single particle, IMO. One should be able to stop when there are (say) less than 16 particles in the subtree

Perhaps PEPC was not intended as a constraining specification, but we need to be clear on what is.

Team Legion

We feel the implementation is underspecified in a few ways:

It isn't clear whether GPUs will be part of the evaluation; Section 7 seems to say no, section 8 seems to say GPUs could be a stretch goal. We quite strongly think that GPUs should be required, as that is where all of the compute power in a system (even a one node system) and the interesting performance engineering challenge is today. Targeting CPUs only doesn't seem like a relevant problem, though it would be fine to target both for comparison.

- **Team Kassel:** We are planning to implement GPU support for our AMT (Itoyori/FBC) down the road, but it won't be done for this paper. We prefer to keep it CPU-only, but if we decide on including GPUs, it would be great to use CPU as our common ground so everyone can be compared fairly, leaving GPUs as an optional add-on.

Optimized, or at least standardized, CPU and GPU kernels should be provided and required for the evaluation. The goal here is presumably to measure the performance contribution (or penalty) of the runtime systems, not the kernels, so standardized kernels will be necessary to make the comparisons meaningful.

There should be some specification of the range of task granularities that the benchmarking will encompass. We suggest a range of 10us - 10ms — not so fine-grain that tasking systems will be uncompetitive with MPI, and not so coarse that all the runtimes look about the same.
The scale of the benchmarking runs should be specified. We suggest scaling to at least 128 nodes; larger would be fine, too.

We don't have experience with a Barnes-Hut implementation, so we don't have a strong opinion about the algorithmic details. We do agree that if we are going to do this we should select a strategy that will be competitive with good MPI implementations. We don't think it is worthwhile to spend effort comparing AMTs if none of the implementations are competitive with existing MPI implementations.

Since we believe that scaling up to large problem sizes is important, we are in favor of a synthetic input generator to make acquiring arbitrarily large inputs straightforward.

Team HPX

We agree with the Legion Team that we should standardize the computation kernels. There are several optimizations/simplifications in computing the energy equation and also when executing the leapfrog steps. We might even want to look at PEPC to see which optimizations/simplifications are implemented.

Given the trend in this document and the valid claim for compatibility, we also support the Morton Tree.

Regarding the dataset, we believe the data generator is essential for arbitrary scaling. However, some partial results with real-world data might strengthen the paper and would add real-world relevance.

We need to be aware that the bulk of PEPC is written in FORTRAN. Additionally, it relies on MPI collectives (though I'm not sure which ones are used in the part we would compare to). How do we plan to compete with the optimized communication algorithms?

How do we handle the communication backends of our AMTs? E.g., HPX supports MPI and LCI. Since we cannot guarantee the same backends across AMTs, we should use the most mature/fastest variant for each AMT.

Team LANL / Initial Response to Feedback

Note: This was sent around as an E-Mail on 07/17. I just decided to also post the relevant parts here for easier referencing/commenting (and for fixing some typos) ~ Gregor

First of all: Thanks for all your feedback. I went through it and had a rethink about some aspects of the project. I also apologize for the late response (I had a bit a of a medical problem at the beginning of July and am playing catchup ever since).

Going through your feedback, I think overspecification can be a real danger here, especially if it delays our initial implementation efforts. To address this I would propose the following:

- Separate implementation phases: One for the initial implementation, one for streamlining and shared optimizations. This gives us more flexibility to address comparability issues with the actual implementations without trying to predict every single aspect/problem that might arise.
  Please see section 1 below for the phases and the intended deadlines.

- Focus the requirements on what we want to achieve without overspecifying. For example, as long as we have a balanced initial load for the first timestep, we can already perform a fair comparison between the AMTs (regardless of how exactly this load was balanced) for this timestep. Please see section 3 below for the details regarding the requirements and section 2 for the intended metrics / comparisons.

The other major concern is whether we should include GPUs or not. Please see section 4 for our considerations on this one.
In a nutshell: each implementation must have a CPU version for full comparison.
However, if an AMT is compatible, we should explore a GPU version in the first implementation phase. If we think it works out for enough teams after this initial implementation phase we will fully commit to include an additional GPU comparison in the final paper (as it makes the paper a lot better).

Overall, I hope the details in the sections below (at the end of the email) address many of your concerns and questions so far.
Of course, these sections are just a rough draft. I will polish the grammar/wording and details a bit over the next week and update the whitepaper accordingly.
However, I think the draft of the sections gets the idea across even in its current form, hence I wanted to send it out to everyone.

1. Implementation phases:

- Implementation Phase 1 -- Initial Implementation
  - Phase 1.1 Have working Single-Node implementation (or distributed implementation running on at least a single node).
    - High-Level goal: This is to get the project really started for each team.
    - DEADLINE: 09/18/2026
    - Should be available for us to look/build at on Github
  - Phase 1.2 Have working distributed version

        - High-Level goal: By the end of this phase we should have a working, initial version from each time that already runs distributed.
        - DEADLINE: 10/30/2026 (considered end of Implementation Phase 1)
        - Needs to run distributed on CPUs (Opt-in GPU).
- Implementation Phase 2 -- Streamlining / Optimizing / Bugfixing
    - DEADLINE 12/18/2026
    - High-Level goal: We aim to reduce differences between our AMT implementations where necessary (in cases where we can further improve the comparability of our implementations) and share optimization where applicable (for example, compute kernels). Will include initial distributed runs done be us to identify problems and where more streamlining is required.
- Data collection Phase (planned January/Late December)

We should conclude each implementation phase (1.1, 1.2, 2) with a meeting to get on the same page.

2. Metrics:

Primary Metrics:

- Runtime of one time-step (excludes load balancing issues over time). This should be done for the first time-step where everyone should have the exact same, balanced data distribution. This will be the first primary comparison we aim for, as this first timestep will provide us with the simplest (and thus fairest) comparison between our AMTs. We can even limit this to just the calculation of the particle interactions within that timestep for the purpose of this comparison.
- Runtime of the entire simulation (load balancing becomes interesting). This will be the second primary comparison. Here we might deal with more subtle differences between our AMTs/implementations than with the first comparison (due to load balancing/communication over time). However, this is the comparison people are most likely to expect. We would also include the PEPC comparison here.
- Strong scalability (via runtime experiments mentioned above) for a reasonably large dataset
    - Note 1: "reasonably large" is intentionally vague until we determine the final machine that we will use. Currently we are still looking at options here. Ideally, we want a machine where we can go at least beyond 1000 compute nodes. We will provide more specific numbers by implementation phase 2.
- Parallel efficiency
- Runtime of the individual phases/steps of the algorithm for the first timestep (particularly the tree construction runtime and the particle interaction runtime).

Secondary Metrics:

- There are a bunch of secondary metrics that might be interesting:

    - Potentially: Track GFLOP/s by tracking the interactions/s. However I would keep this as an optional idea for now and revisit if we want to track this in Implementation Phase 2.
    - Potentially: Try different numbers of particles per tree-node (leaf), for the first comparison. A lower number of particles per node should decrease the computational efficiency but in turn make overheads in the runtime itself more noticeable as we deal with a deeper tree and more tasks. We will determine whether to use this during our initial runs in Implementation Phase 2 (as it might not be necessary/interesting depending on the results).
    - Note: Feel free to propose additional interesting metrics by the beginning of Implementation Phase 2.

Excluded Metrics:

- We plan to exclude I/O times from this comparison
- We do not plan for code-complexity comparisons at this point (it will be hard to stay within the page limit at SC as it is). However, once we have the implementations, this can of course be a follow-up project if there is interest.

3. Requirements:

Basic Requirements

- Each implementation needs to implement the Barnes Hut algorithm for gravitational nbody problems (see whitepaper).
- Each implementation needs to be compatible with the datasets generated by our data-generator (see whitepaper).
- Each implementation needs to track the energy conservation across the simulation to ensure correctness (see whitepaper).
- Each simulation should be capable of exporting a snapshot for a Paraview visualization of the final state (ensure correctness, plus we should have some visualizations).

Timer requirements:

- Different phases (input, tree construction, particle interactions + energy calculations, output, load balancing) should be tracked by timers.
    - Why: This allows us to compare the implementations in more detail. This also allows us to exclude the I/O times. Specifically, we can compare a single time-step (or even just the calculation of the particle interactions) for a perfectly balanced initial iteration, thus reducing the impact of the differences between our individual implementations for one of our benchmarks.
    - Note: Timers should track walltime of the individual phases/steps of the algorithm, and thus might introduce synchronization points due to that.

- Individual Timers can be turned off by parameter.
    - Why: This is to allow interleaving between phases as a possible optimization. For example, for the full simulation run, we do not need to track the separate runtimes for tree construction, interaction/energy calculations and rebalancing (though the input time should still be tracked to exclude it from the total runtime). After all, we are all about task-graphs and interleaving tasks/communication.

Data-Distribution, Data-structure and Communication requirements:

- Particles should be distributed across all compute nodes (no replication of the entire particle set for each compute node!).
    - Why: For competitive performance, scalability and comparability to existing MPI implementations, the particles need to be distributed across the compute nodes.
    - Note 1: We plan to exclude the input time for our runtime comparisons (as we think that parallel I/O performance is a topic of its own).
    - Note 2: This specifically means that it is allowed to read the dataset on one node and then distribute it across all compute nodes as an initial step. This can potentially save us implementation effort by foregoing parallel I/O considerations (though if a team wants to implement a parallel input approach they still can).
- Each implementation is responsible to have a perfectly balanced initial state for the first timestep (each compute node should have the same number of particles or as close to it as possible)
    - Why: This allows to perform near-perfect runtime comparisons for the first particle interactions iteration (and the first time-step as a whole). Hence this is handy for a comparison between our AMTs whereof slight differences between the implementations (that will probably creep up) are simply irrelevant.
    - Note1: The implementation details for how this balancing is to be done is up to the teams. We encourage a Morton ID based approach, however, as long as we all end up with a balanced initial state the runtime comparison of the first timestep, the comparison should be sound.
    - Note 2: Specifically, we decided to drop any specific requirement to use a hashable oct-tree (though teams are of course welcome to use this approach).
- Load balancing should be configurable (on/off, rebalancing frequency).
    - Why: Longer runs will require load balancing for competitive performance when compared to existing MPI implementations. Yet, comparisons between the AMTs themselves might still be interesting without the load balancing enabled for a few timesteps (mostly to exclude differences caused by different load balancing implementations). Hence it is best to keep this configurable, thus allowing us various comparison options for the eventual paper (or follow-up papers).
    - Note: Every team should try to implement to node-balancing to think works best for their individual implementation. It will definitely be used for the comparison against PEPC.
- If an AMT supports multiple networking backends we will use the one that works best.

    - Why: This keeps the runtime data closer to what people would see in real-world applications. For example, HPX has an MPI-based backend and an LCI-based backend. While using the MPI-based backend sounds like a good idea for a comparison to some MPI-based code, the backend is rather old and people are more likely to use the LCI networking backends within HPX nowadays as it provides better performance/scalability.
    - Note: The same goes for any other unique communication features/optimizations each AMT has. We aim for the best distributed performance so when a feature makes sense, it should be used. While this makes comparisons more annoying, it better reflects the real-world performance people can expect from the AMTs.

Compute Kernel Requirements (CPU):

- Maximum number of particles for each tree node should be configurable at either compile-time or as a runtime parameter at the beginning of the simulation. This number will stay the same for the duration of the simulation.
    - Why: Using just one particle per compute node allows us to put maximum stress on our AMTs and can be useful for the comparison between our AMTs -- however, the computations become inefficient.
      Using more particles per node will thus be better for a comparison with PEPC. Keeping this configurable gives us options here, depending on what we want to compare.
    - Note: This can also be a potential avenue for a GPU implementation as we can increase the number of particles per tree node to a sufficient size for GPU kernels.
- Use unoptimized compute kernels for initial implementation
    - Why: We plan to streamline the actual compute kernels during the Implementation Phase 2 so that every team uses the same compute optimizations where possible. Hence premature optimization here is not required beforehand.

4. GPU vs CPU considerations:

- We agree that using GPUs would improve the paper. However, given that not all AMTs support GPUs we cannot add this as a hard requirement for each implementation. Instead we propose the following:
    - Each implementation MUST support a CPU-only configuration to allow for full comparison of all AMTs and the MPI-based comparison code (PEPC) during the data-collection phase of the project.
    - Each implementation SHOULD explore a GPU-acceleration implementation that can be activated at compile-time. A likely avenue for this is to exploit the configurable number of particles in each leaf tree-node to accelerate the interaction calculations. This exploration should be done in phase 1 (by the end

of 1.2). However, if this is not feasible for an AMT or for a team (due to time constraints), this exploration may be dropped in favor of focusing on the CPU-only implementation (without any GPU option).

- If two or more implementations reach a feasible GPU implementation by the end of phase 1.2, we will include this in the rest of the project.
  - Note 1: In this case, it would of course also be part of the streamlining effort in phase 2 to improve our comparability by sharing compute kernel implementations.
  - Note 2: Personally, I would be pretty excited to experiment with this for the HPX implementation so that would be one implementation already.
- We should aim to use plain CUDA for any GPU implementation. If an AMT contains integration features / optimizations for CUDA, these should be used where applicable.
  - Why: NVIDIA GPUs and CUDA seem to enjoy the broadest support across our AMTs and are probably of the greatest interest to any reader. While performance portability frameworks such as Kokkos/SYCL would be interesting as well, they are less commonly used and are not as well supported by our AMTs overall. Integration features should be used (if any) as we should stay close to real world performance for each AMT instead of artificially limiting us for the sake of a perfect comparison.
  - Note: The GPU partition of Perlmutter might be a good target for any potential GPU comparison.

Team PaRSEC

We agree with the thoughtful comments and suggestions from the participating teams. From the PaRSEC perspective, we plan to explore implementations of the Barnes–Hut benchmark using all three programming interfaces in PaRSEC: PTG, DTD, and TTG. At a minimum, we expect to provide DTD and TTG implementations.

Comparing multiple implementations within the same runtime may also provide useful insight into how different task-graph abstractions affect programmability, task granularity, dependency management, communication, and performance. However, we agree that the benchmark specification should define common algorithmic requirements, correctness criteria, input datasets, and timing boundaries while still allowing each AMT to use the programming model and internal data representation most natural to its runtime.

It mentions PEPC as a scalable MPI/OpenMP implementation, but a smaller and easier-to-understand reference implementation would also be helpful for development and verification.